a type guard, and inside the `if` block protected by the `in` statement, TypeScript works on a list of available attributes based on the elements of the union type that contain that attribute. If multiple component types contain the attribute—both `Square` and `Triangle` have a `width` attribute, for example—then inside the type guard, TypeScript will only allow you to access an attribute that is available in *all* the component types with that attribute. In our case, we now have a bug because both `Square` and `Triangle` would go down the second branch, but the formula is for square, which should give you a sense of the limitations of this mechanism.

An important extension here is that we don't need to specify that the `else` branch is a `Circle`; TypeScript will infer that the `else` branch is all the types in the union type not covered by the `if` branch.

Admittedly, the `in` operator can be tricky. Alternatively, you can write your own functions and have TypeScript treat them as type guards. For example:

```
function isCircle(shape: Square | Triangle | Circle): shape is Circle {
  return (shape as Square).radius !== undefined
}
```

The syntax for this is, frankly, a little verbose and weird. The function takes an argument, which would typically have the entire union type as its type (otherwise you wouldn't be able to call the function with any potential member of the union type). The return value of the function is what TypeScript calls a *type predicate* and the syntax is `<variable> is <type>`; the variable name has to match the name of the argument to the function.

The body of the function is whatever you want, basically, as long as the function returns true when you can show that the argument is of the type needed. The recommended way to do that is to check for the existence of an attribute that is in that class. In our case, we're checking to see if the shape is a circle by testing for the existence of a `radius` attribute.

We can then use this function as we've been using our other type guard: